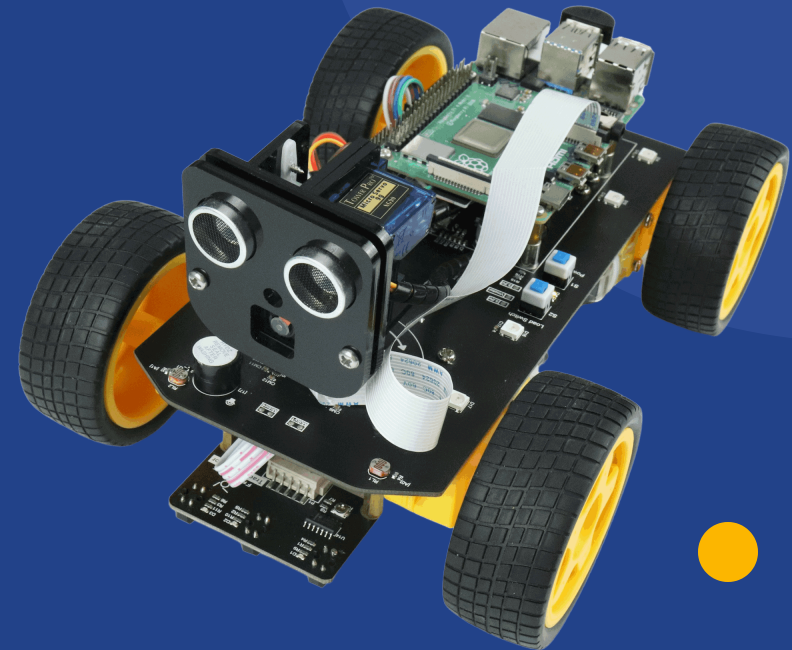




LESSON 8 – MULTIPLE FACE DETECTION

Machine Learning with Raspberry Pi & Smart Car

Find **everyone** in view — then **lock onto one** and keep following that one person smoothly, even as other faces come and go.



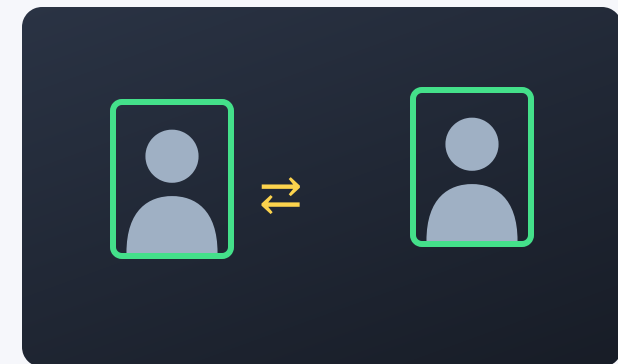


LESSON 8 – MULTIPLE FACE DETECTION

What if two people are in view?

Lesson 7 followed the **biggest** face. With a crowd, the car can flip back and forth.

- Lesson 7 followed the **biggest** face – fine for one person.
- With **two people**, the car flips back and forth – jumpy and confusing.
- Today: detect **everyone**, but **lock onto one** target and stick with it.



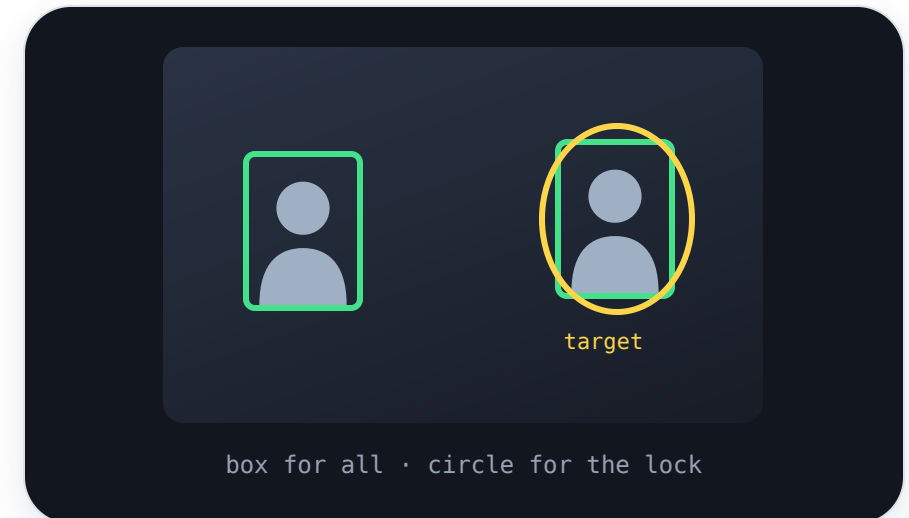
which face do I follow?

LESSON 8 – MULTIPLE FACE DETECTION

Today's goal: detect all, follow one

Build it by **reusing all of Lesson 7** – the headline skill today is **code reuse**.

- 1 Draw a box around **every** face.
- 2 Keep a **target lock** on one face.
- 3 Follow that target **smoothly**.





LESSON 8 – MULTIPLE FACE DETECTION • THE KEY IDEA

Reuse Lesson 7 by importing it

`import lesson_7_face_tracking as face` brings in **all** of Lesson 7's functions.

- Call `face.read_frame()`, `face.detect_faces()`, `face.track_face()` ...
- Importing runs L7's **top part** (imports, settings, functions)...
- ...but **not** its `if __name__ == "__main__"` block – so L7 doesn't auto-start.



WHY THE GUARD MATTERED

L7's `__main__` guard means importing it reuses the **toolbox** without running the program.

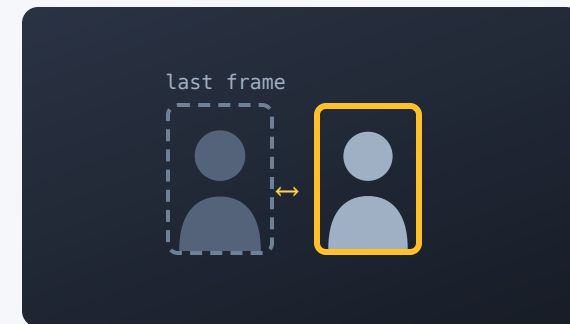


LESSON 8 – MULTIPLE FACE DETECTION • THE KEY IDEA

Target lock: a little memory

"Closest to last time" gives the program a memory – that's what stops the jumping.

- 1 **First frame:** pick the **biggest** face as the target.
- 2 **After that:** pick the face **closest** to last frame's target.
- 3 Nobody close enough → fall back to the **biggest** face.



within LOCK_DISTANCE → keep lock



BUILD THE FILE • CHUNK 1 / 8

Start the file: imports

Just `cv2` (to draw) and Lesson 7 (for everything else).

- `cv2` is used here just to **draw** on the picture.
- The nickname `face` keeps the calls short.
- This import is the **whole point** of the lesson.

```
lesson_8_multiple_face_detection.py

"""Lesson 8: Multiple Face Detection."""

import cv2                # OpenCV: here just to draw on the picture

# Import all of Lesson 7's functions under the short name "face".
# Importing runs L7's top part but NOT its __main__ block, so it
# doesn't start running on its own. Keep this file beside L7.
import lesson_7_face_tracking as face
```



BUILD THE FILE • CHUNK 2 / 8

Settings

A window name, two colors, and the **lock distance**.

- **Green** for every face, **yellow** for the target.
- OpenCV colors are (Blue, Green, Red) – **not** RGB.
- `LOCK_DISTANCE` = how far the target can move and stay locked.

```
lesson_8_multiple_face_detection.py

WINDOW_NAME = "Lesson 8 - Multiple Face Detection"
BOX_COLOR = (0, 255, 0)           # green
TARGET_COLOR = (0, 255, 255)      # yellow
LOCK_DISTANCE = 85                # pixels: how far the target
                                  # can move and stay locked
```



BUILD THE FILE • CHUNK 3 / 8

face_center() & center_distance()

The middle of a face box, and the straight-line distance between two faces.

- `face_center` → the **middle point** of a face box.
- `center_distance` → how **far apart** two faces are.
- It's the **Pythagorean theorem** – $a^2+b^2=c^2$. Hello, math class.

lesson_8_multiple_face_detection.py

```
def face_center(face_box):  
    """Return the middle point of a face box."""  
    x, y, w, h = face_box  
    return int(x + w / 2), int(y + h / 2)  
  
def center_distance(first_face, second_face):  
    """Return the distance between two face centers."""  
    first_x, first_y = face_center(first_face)  
    second_x, second_y = face_center(second_face)  
    return ((first_x - second_x) ** 2  
            + (first_y - second_y) ** 2) ** 0.5
```



BUILD THE FILE • CHUNK 4 / 8

choose_target(): follow the same person

Returns two things: the face **and** a label we show on screen.

- No target yet → start with the **biggest** face.
- Have a target → pick the **closest** face to last frame.
- Too far from any face → restart with the **biggest**.

```
lesson_8_multiple_face_detection.py

def choose_target(faces, previous_target):
    """Choose one face to follow, reusing the previous target."""
    if len(faces) == 0:
        return None, "none"
    if previous_target is None:
        return face.biggest_face(faces), "largest"

    closest_face = min(faces,
                       key=lambda f: center_distance(f, previous_target))
    if center_distance(closest_face, previous_target) <= LOCK_DISTANCE:
        return closest_face, "locked"
    return face.biggest_face(faces), "largest"
```



BUILD THE FILE • CHUNK 5 / 8

draw_faces(): mark all, highlight the target

The takeaway: **boxes for all, a circle for the target.**

- **Green box** around every face.
- A **yellow circle + label** on the one we follow.
- Don't dwell on the drawing args.

```
lesson_8_multiple_face_detection.py

def draw_faces(frame, faces, target, target_mode):
    """Draw every face and highlight the target face."""
    for face_box in faces:
        x, y, w, h = face_box
        cv2.rectangle(frame, (x, y), (x + w, y + h), BOX_COLOR, 2)

    if target is not None:
        x, y, w, h = target
        center = face_center(target)
        cv2.circle(frame, center, int((w + h) / 4), TARGET_COLOR, 2)
        cv2.putText(frame, "target: " + target_mode,
                    (x, min(frame.shape[0] - 10, y + h + 18)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, TARGET_COLOR, 1, cv2.LINE_AA)
```



BUILD THE FILE • CHUNK 6 / 8

draw_detection_summary(): on-screen status

A small line at the top: how many faces, and what the target is doing.

- Shows **face count** and the target **mode**.
- Great for debugging – see `locked` vs `largest` **live**.
- Drawn near the **top** of the picture.

```
lesson_8_multiple_face_detection.py

def draw_detection_summary(frame, faces, target, target_mode):
    """Show a short summary for this video frame."""
    if target is None:
        summary = "faces: 0  target: none"
    else:
        summary = ("faces: " + str(len(faces))
                  + "  target: " + target_mode)
    cv2.putText(frame, summary, (10, 22),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, TARGET_COLOR, 2, cv2.LINE_AA)
```



BUILD THE FILE • CHUNK 7 / 8

loop(): reuse + lock + draw

Every `face.xxx()` call is a **Lesson 7** function doing the heavy lifting.

- Reuse L7 for the **camera, detection, head movement**.
- New: keep a `previous_target` so the lock **carries over**.
- Press `q` to quit.

```
lesson_8_multiple_face_detection.py

def loop():
    """Mark all faces, keep a target lock, show the video."""
    previous_target = None
    while True:
        frame = face.read_frame()          # from Lesson 7
        if frame is None:
            break
        faces = face.detect_faces(frame)    # L7: find EVERY face
        target, target_mode = choose_target(faces, previous_target)
        previous_target = target
        draw_faces(frame, faces, target, target_mode)
        draw_detection_summary(frame, faces, target, target_mode)
        face.track_face(target)             # L7: turn toward target
        cv2.imshow(WINDOW_NAME, frame)
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
```



BUILD THE FILE • CHUNK 8 / 8

The main block: borrow `setup()` & `destroy()`

We even reuse L7's `setup()` and `destroy()` – the connection code, for free.

- `face.setup()` – connect + load the face detector.
- `loop()` – our own loop, defined above.
- `face.destroy()` – close the sockets and window.

```
lesson_8_multiple_face_detection.py

if __name__ == "__main__":
    try:
        face.setup()      # L7: connect + load the detector
        loop()            # our own loop, defined above
    except KeyboardInterrupt:
        pass
    finally:
        face.destroy()    # L7: close the sockets and window
```



LESSON 8 – MULTIPLE FACE DETECTION

We built the whole file

Most of the program is **reused** from Lesson 7 – that's the power of good functions.



lesson_8_multiple_face_detection.py

`import lesson 7` → `settings` → `face_center()` → `center_distance()` → `choose_target()` → `draw_faces()` → `draw_detection_summary()` → `loop()` → `main`



Almost no new hardware code: the camera, detection, head movement, and connections all came from Lesson 7 – we just added a `target lock` on top.



LESSON 8 – MULTIPLE FACE DETECTION

Run it

Keep `lesson_8...py` in the **same folder** as `lesson_7_face_tracking.py`.

Step 1 – on the Pi, start the car's server

```
● ● ● Pi terminal

cd Smart-Car-Level-3/Code/Server
sudo python3 main.py
```

Step 2 – on the Pi or your laptop, run it

```
● ● ● terminal

cd Smart-Car-Level-3/Code/User
python lesson_8_multiple_face_detection.py 192.168.1.50
```



Get a **friend in frame** – watch the yellow target stay locked. Use `127.0.0.1` on the Pi.



Green boxes mark everyone; the yellow circle follows your target. Press `q` to quit.



LESSON 8 – MULTIPLE FACE DETECTION

Try it yourself

`LOCK_DISTANCE` is the fun dial – feel **sticky** vs. **switchy** tracking.

- Change `LOCK_DISTANCE` – bigger = **stickier**, smaller = switches easily.
- Change `TARGET_COLOR` to your favorite (remember: **B, G, R**).
- Add your own text to the **summary** line.

CHALLENGE

Lock onto the closest person

Make the target the face **closest to the camera** (the biggest box) instead of nearest-to-last.



LESSON 8 – MULTIPLE FACE DETECTION

Common issues

Most new issues are the missing neighbor-file or a `LOCK_DISTANCE` that needs tuning.

**ModuleNotFoundError: lesson_7...**

The two files must be in the **same folder**.

**Target jumps between people**

Raise `LOCK_DISTANCE`.

**Lock "sticks" to the wrong face**

Lower `LOCK_DISTANCE`.

**No video / refused**

Same as L7: start `sudo python3 main.py` first; check the IP.

**Same neighbor**

Vision and connection issues are **identical to Lesson 7**. The new failure is just the two files not sitting together.



LESSON 8 – MULTIPLE FACE DETECTION

You finished the code curriculum!

You **reused an entire program** by importing it – real software engineering.

- ✓ Reused all of Lesson 7 by **importing** it
- ✓ Detected **every** face and boxed them all
- ✓ Added **target lock** to follow one person smoothly



UP NEXT

Lesson 9 Final Project Workshop

That's the **last code lesson**. Next, you'll **build your own idea** – bring it to the workshop.



• COACH ONLY • HIDDEN SLIDE

End-of-class checklist

Don't present this slide. Run through it right after the last code lesson.

1

Post the after-class recap

Summarise what you covered on the class page so students can catch up.

2

Log absent students

Post anyone who missed class in the **#attendance** Discord channel.

3

Tee up the Final Project Workshop

This was the **last code lesson**. Point students toward **project ideas** for **Lesson 9**.

EVERY CLASS

✓ Post a class announcement

✓ Introductions if needed

✓ Note absences in #attendance